THE ADVANCED PYTHON FULL-STACK ENGINEER ROADMAP

A practical path from foundations to professional software engineering

Frontend HTML • CSS • JavaScript • TypeScript • React	**Python backend** Python • Django • Django REST Framework • FastAPI
Data SQL • PostgreSQL • Redis • data modelling	**Engineering** Testing • Security • Git • Docker • Cloud • System Design

Recommended direction: React + TypeScript on the frontend, Python + Django/DRF for the main backend, PostgreSQL for data, and FastAPI when you need focused high-performance APIs or AI/ML services.

This roadmap is deliberately staged. Learn one main stack, build complete products, then add advanced tools when a real project needs them.

Designed for a CSE student who wants strong software engineering fundamentals and the ability to build AI-enabled products.

1. Is Python really suitable for full-stack development?

Yes. Full-stack means you can build the browser interface, server-side logic, data layer, tests and deployment. It does not mean one language must do everything. A common professional architecture uses JavaScript/TypeScript in the browser and Python on the server.

What Python is excellent at

• Backend business logic, authentication workflows, database-backed applications and REST APIs.
• Django provides an integrated toolkit: routing, ORM, migrations, admin, forms and security features.
• FastAPI is excellent for typed APIs, async endpoints and serving AI/ML model functionality.
• Python has a broad ecosystem for data science, machine learning, automation and AI integration.

What Python does not replace

• HTML defines page structure; CSS controls presentation and responsive layouts.
• JavaScript runs in browsers and handles interaction. TypeScript adds static typing to JavaScript projects.
• React is a frontend library, not a backend replacement.
• SQL remains important. An ORM does not remove the need to understand queries, joins, indexes and transactions.

When should you choose another backend language?

Choice	Good fit	Advice
Python + Django/FastAPI	Web products, APIs, AI/ML integration, rapid development	Recommended primary path for this roadmap.
Java + Spring Boot	Large enterprise systems, JVM ecosystem, strongly structured backend teams	A valuable second backend stack; do not learn in parallel at the beginning.
Node.js + TypeScript	Teams that want TypeScript across frontend and backend, event-driven APIs	Good alternative, but not necessary if Python is your chosen backend.
Kotlin + Spring Boot	JVM backend and Kotlin-focused engineering	Useful if you later want to deepen your Kotlin path.

Decision: Do not switch away from Python just because it is not used for browser frontend. Keep Python as your main backend language and learn JavaScript/TypeScript for the frontend.

2. Target architecture

Browser (React + TypeScript) → HTTPS/JSON API → Python application (Django/DRF) → PostgreSQL. Redis can support caching or background jobs. A separate FastAPI service can expose an AI model when the project needs it. Docker packages services consistently; a cloud platform runs them.

3. Foundations: web, HTML, CSS and JavaScript

Phase 0 — How the web works (3–5 days)

• Browser, client, server, DNS, domain, hosting, ports and localhost.
• HTTP vs HTTPS; request/response; methods; status codes; headers; cookies; sessions.
• JSON, URL paths, query parameters, request body and browser developer tools.
• Basic command line: navigate folders, run programs, inspect processes and ports.

Phase 1 — HTML5 (1–2 weeks)

• Document structure, headings, paragraphs, links, images, lists and tables.
• Semantic elements: header, nav, main, section, article, aside and footer.
• Forms: labels, input types, textarea, select, validation and accessible controls.
• Accessibility: alt text, keyboard navigation, focus order and meaningful labels.
• SEO foundations: page titles, descriptions, semantic structure and metadata.

Phase 2 — CSS3 (2–3 weeks)

• Selectors, cascade, specificity, inheritance, box model and units.
• Typography, color, spacing, borders, shadows, backgrounds and custom properties.
• Flexbox, CSS Grid, positioning, stacking context and overflow.
• Responsive design, mobile-first CSS, media queries and fluid sizing.
• Transitions, transforms and simple animations; avoid overusing effects.
• Build a responsive portfolio, landing page and admin dashboard layout.

Phase 3 — JavaScript, then TypeScript (4–6 weeks)

• Values, primitive/reference types, operators, conditions, loops and functions.
• Arrays, objects, map/filter/reduce, destructuring, spread/rest and modules.
• Scope, closures, this, prototypes and the event loop at a practical level.
• DOM selection, events, form handling, validation and browser debugging.
• Promises, async/await, fetch, JSON, errors, timeouts and API interaction.
• Browser storage, cookies, CORS, same-origin policy and basic web security.
• TypeScript: types, interfaces, unions, generics, narrowing and tsconfig basics.

Checkpoint before React

Build a responsive multi-page website with a working form, client-side validation, accessible navigation and an API-powered feature. You should be able to explain every line you wrote.

4. Frontend engineering with React + TypeScript

• React mental model: components, JSX, props, state, rendering and reconciliation basics.
• Component design: composition, reusable UI, controlled inputs and lifting state.
• Hooks: useState, useEffect, useContext, useRef; understand dependencies and cleanup.
• Lists and keys, conditional rendering, forms, validation and error boundaries.
• Routing, nested routes, route parameters and protected route patterns.
• API integration: loading, success, empty and error states; retries and cancellation.
• State strategy: local state first; Context for modest shared state; learn a state library only when needed.
• Styling approach: CSS modules or a consistent utility/component system.
• Frontend testing: component tests, user interactions and end-to-end browser tests.
• Performance: code splitting, lazy loading, memoization only when measured, image optimization.
• Accessibility: semantic controls, keyboard support, focus management and contrast.

Frontend deliverables

• Portfolio site with responsive layout.
• React dashboard that consumes a public API.
• Authenticated app UI with forms, pagination, filters and role-aware navigation.

5. Python engineering foundations

Do not rush directly into Django. Learn Python well enough to write maintainable modules, reason about data, debug errors and test your code.

Topic	Detailed concepts to master
Core language	Variables, objects, numeric/string/boolean types, None, operators, conditions, loops, functions, scope, imports and modules.
Collections	Lists, tuples, sets, dictionaries, mutability, identity vs equality, slicing, comprehensions, sorting and nested structures.
Functions	Parameters, defaults, keyword-only args, *args/**kwargs, return values, lambda, higher-order functions and recursion.
OOP	Classes, objects, self, constructors, instance/class/static methods, inheritance, composition, polymorphism, properties and dunder methods.
Errors	Exceptions, custom exceptions, try/except/else/finally, raising errors and designing clear failure paths.
Intermediate	Iterators, generators, decorators, context managers, dataclasses, enums, type hints and protocols at an introductory level.
Professional workflow	venv, pip, requirements/pyproject, formatting, linting, debugging, logging and documentation.
Testing	pytest basics, assertions, fixtures, parametrization, mocking and testable design.

Python checkpoint

Create a command-line task or expense tracker using modules, classes, file/database persistence, exception handling and pytest. Use Git from the first day.

6. Main backend path: Django + Django REST Framework

Learn Django first as the primary full-stack backend. It gives you a complete application structure and teaches the request lifecycle, database modelling and secure user workflows.

Django core

• Project vs app, settings, URL configuration, views, templates and request/response objects.
• Models, fields, relationships, migrations, QuerySets and Django ORM.
• Admin customization, forms, validation, file uploads and static/media files.
• Authentication, users, password handling, sessions, groups, permissions and authorization.
• Middleware, signals (use sparingly), custom management commands and configuration.
• Database transactions, select_related/prefetch_related and avoiding N+1 query patterns.
• Production settings, allowed hosts, secret management, static assets and logging.

Django REST Framework (DRF)

• Serializers, validation, ModelSerializer and nested relationships.
• API views, generic views, viewsets, routers and permissions.
• Authentication choices, pagination, filtering, search, ordering and throttling.
• Consistent response formats, status codes, error handling and API versioning basics.
• OpenAPI/Swagger documentation and API testing.
• CORS configuration and safe frontend-backend integration.

FastAPI — learn after a solid Django project

• Path/query/body parameters, Pydantic models, validation and dependency injection.
• Async endpoints and where async helps; do not assume every endpoint must be async.
• Authentication, middleware, error handling, background tasks and OpenAPI docs.
• SQLAlchemy or SQLModel basics, migrations with Alembic and test clients.
• Use FastAPI for focused services, integrations or AI/ML inference APIs when it fits.

Backend rule

Do not try to master Django, Flask and FastAPI simultaneously. Learn Django + DRF deeply first. Learn FastAPI as a second tool for a clear use case. Flask is optional; it is not a required step.

7. Database and data engineering fundamentals

• Relational design: entities, attributes, primary keys, foreign keys and constraints.
• SQL: SELECT, INSERT, UPDATE, DELETE, WHERE, ORDER BY, GROUP BY and HAVING.
• JOINs: INNER, LEFT and self joins; subqueries and common table expressions.
• Relationships: one-to-one, one-to-many and many-to-many; normalization basics.
• Indexes, query plans, EXPLAIN, transactions and ACID properties.
• PostgreSQL types, constraints, views, JSONB basics, backups and restore.
• ORM trade-offs: understand the SQL generated by Django ORM or SQLAlchemy.
• Redis fundamentals: cache, TTL, invalidation and rate-limit use cases.

8. Professional software engineering

Git and collaboration

• Git status/add/commit/log/diff, branches, merge, rebase basics and conflict resolution.
• GitHub repositories, README, issues, pull requests, code review and release tags.
• Commit small logical changes; use meaningful messages; never commit secrets.

Testing and quality

• Unit, integration, API and end-to-end tests; know what each level proves.
• pytest, Django test tools, React Testing Library and Playwright/Cypress concepts.
• Test success cases, invalid input, permission failures, edge cases and regressions.
• Formatting, linting, type checking, code review and readable documentation.

Security — essential, not optional

• Authentication proves identity; authorization decides what the identity may do.
• Password hashing through trusted framework tools; never store plaintext passwords.
• Understand SQL injection, XSS, CSRF, CORS, SSRF and insecure direct object references.
• Validate input on the server; never trust frontend checks as security controls.
• Secure cookies, HTTPS, secret management, least privilege and dependency updates.
• Rate limiting, audit logs, safe file uploads and careful error messages.
• Use established payment providers and never store raw card details.

API and application design

• REST resource design, idempotency, pagination, filtering and versioning.
• Validation boundaries, service layer, data access layer and separation of concerns.
• Configuration by environment; structured logs and correlation/request IDs.
• Background jobs, queues, retries, idempotent task handling and scheduled work.
• Caching strategy, cache invalidation, file/object storage and email notifications.

9. Advanced topics: from developer to engineer

Area	Advanced topics
Architecture	Layered architecture, modular monolith, domain-driven design basics, dependency inversion, design patterns and refactoring.
Concurrency	Processes vs threads, async I/O, race conditions, locks, task queues and worker processes.
Scalability	Caching, database indexing, read replicas concepts, load balancing, horizontal scaling and bottleneck analysis.
Distributed systems	Queues, eventual consistency, retries, timeouts, circuit breakers, idempotency and failure modes.
System design	Requirements, APIs, data model, capacity estimates, trade-offs, reliability and observability.
Performance	Profiling, query optimization, memory use, frontend performance and load testing.
Cloud & DevOps	Linux, DNS, TLS, reverse proxy, containers, CI/CD, managed databases, logs, metrics and backups.
Reliability	Health checks, graceful shutdown, deployment rollback, incident debugging and recovery planning.

Learn advanced topics through project problems. For example, add Redis only when you can explain what is slow and why caching is appropriate.

10. Deployment, Docker and operations

• Linux: files, permissions, environment variables, processes, services, ports and logs.
• Networking basics: DNS, TCP, HTTP, TLS, reverse proxies and firewall concepts.
• Deployment: build frontend, configure backend, provision database, run migrations and serve static assets.
• Docker: images, containers, Dockerfile, volumes, networks and Docker Compose.
• CI/CD: automated linting, tests, builds and deployment pipeline concepts.
• Production: secrets, HTTPS, backups, monitoring, error tracking and rollback plan.
• Cloud: learn one provider's fundamentals; understand compute, storage, managed database, networking and IAM.

11. AI-enabled full-stack development (optional specialization)

This is a useful specialization if you want to combine software development with AI/ML. First become capable of building a normal reliable web application; then integrate AI.

• Call model APIs securely from the backend; keep provider keys on the server.
• Streaming responses, conversation persistence, usage limits, retries and cost monitoring.
• Embeddings, vector search, retrieval-augmented generation (RAG) and evaluation basics.
• File ingestion, chunking, metadata, access control and source citations.
• Background processing for long-running tasks; do not block ordinary web requests.
• Treat model output as untrusted input; validate actions and enforce permissions in backend code.
• Deploy model-serving or ML inference separately when dependencies or scaling needs justify it.

12. Project ladder — prove each skill

Project	Required features / skills demonstrated
1. Portfolio	Responsive HTML/CSS, accessible design, JavaScript interactions, deployed site.
2. Task manager	React UI, Python API, CRUD, PostgreSQL, validation and tests.
3. Notes app	Authentication, per-user data permissions, search, pagination and secure file handling.
4. Expense tracker	Relational data model, reports, date filtering, charts and export.
5. E-commerce system	Product catalogue, cart, orders, admin, stock handling, mock payment workflow and transactional logic.
6. Capstone: internship portal	Student/recruiter roles, profiles, jobs, applications, search, notifications, audit trail, admin, tests and deployment.
7. AI feature extension	Add document Q&A; or AI assistant using a backend model API, RAG, access control and usage limits.

Capstone quality bar

• A clear README: problem, features, architecture diagram, setup, environment variables and screenshots.
• A live demo with realistic seed data and a test account that contains no sensitive real data.
• Automated tests for core workflows, invalid input and authorization boundaries.
• Database migrations, backups, structured logs and documented deployment steps.
• A short design note explaining trade-offs, limitations and what you would improve next.

13. Suggested 40-week learning plan

A sample plan for roughly 12–15 focused hours per week. Extend it if your college workload is heavy. The goal is demonstrated ability, not racing through dates.

Weeks	Main focus	Evidence / deliverable
1–2	Web foundations + HTML	Semantic multi-page site
3–5	CSS + responsive design	Mobile-friendly portfolio
6–10	JavaScript + TypeScript basics	Interactive app with API calls
11–14	React + TypeScript	Dashboard frontend
15–18	Python foundations + testing	CLI app, modules and tests
19–24	Django + PostgreSQL	Authenticated CRUD application
25–28	DRF + API integration	Documented, tested REST API
29–31	Security + quality + Git workflow	Permission tests and clean repository
32–35	Docker + deployment + operations	Live deployed application
36–40	Capstone + polish	Portfolio-grade full-stack product

Weekly study loop

• 30% learn concepts and read official documentation.
• 50% implement features yourself and debug failures.
• 10% write tests, refactor and review code.
• 10% explain what you built in a README, diagram or short technical note.

Do not measure progress only by videos completed. Measure it by features you can build from a blank folder, bugs you can diagnose and design decisions you can explain.

14. What makes someone advanced?

• You can translate an unclear requirement into small technical tasks.
• You can design a database and API before writing a large amount of code.
• You can write readable, testable code and explain its trade-offs.
• You can find bugs using logs, tests, debuggers and profiling tools.
• You understand security and never rely on frontend-only permission checks.
• You can deploy, monitor, back up and recover your application.
• You can read documentation, learn unfamiliar tools and collaborate through code review.
• You can maintain a project after the first demo instead of abandoning it.

There is no final state called a perfect developer. Professional growth is continuous. Aim to become reliable, adaptable and capable of shipping software that real people can use.

15. Official resources — use these as the source of truth

Web foundations

MDN Learn Web Development: https://developer.mozilla.org/en-US/docs/Learn_web_development
MDN JavaScript Guide: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
React Learn: https://react.dev/learn

Python and backend

Python Tutorial: https://docs.python.org/3/tutorial/
Django documentation: https://docs.djangoproject.com/
Django REST Framework: https://www.django-rest-framework.org/
FastAPI documentation: https://fastapi.tiangolo.com/
pytest documentation: https://docs.pytest.org/

Database, tools and operations

PostgreSQL tutorial: https://www.postgresql.org/docs/current/tutorial.html
Git book: https://git-scm.com/book/en/v2
Docker getting started: https://docs.docker.com/get-started/
OWASP Top 10: https://owasp.org/www-project-top-ten/
Playwright: https://playwright.dev/docs/intro

16. Final stack checklist

• Frontend: HTML5, CSS3, JavaScript, TypeScript, React, responsive design and accessibility.
• Backend: Python, Django, Django REST Framework; FastAPI as an additional tool.
• Database: SQL, PostgreSQL, data modelling, indexes, transactions and ORM.
• Engineering: Git/GitHub, pytest, API testing, frontend testing, debugging and documentation.
• Security: authentication, authorization, validation, CSRF, XSS, SQL injection, secrets and HTTPS.
• Operations: Linux, Docker, deployment, CI/CD basics, logging, monitoring and backups.
• Advanced: architecture, caching, queues, concurrency, system design and performance.
• Portfolio: at least two smaller deployed apps and one polished end-to-end capstone.

Your first action this week

Start with HTML structure and basic CSS. Build a one-page personal portfolio. Do not jump straight into Django before you understand how a browser page, form and HTTP request work.

Keep the stack focused: React + TypeScript + Python/Django + PostgreSQL. Add FastAPI, Redis, Docker and cloud in stages. Build first; add tools when they solve a real problem.